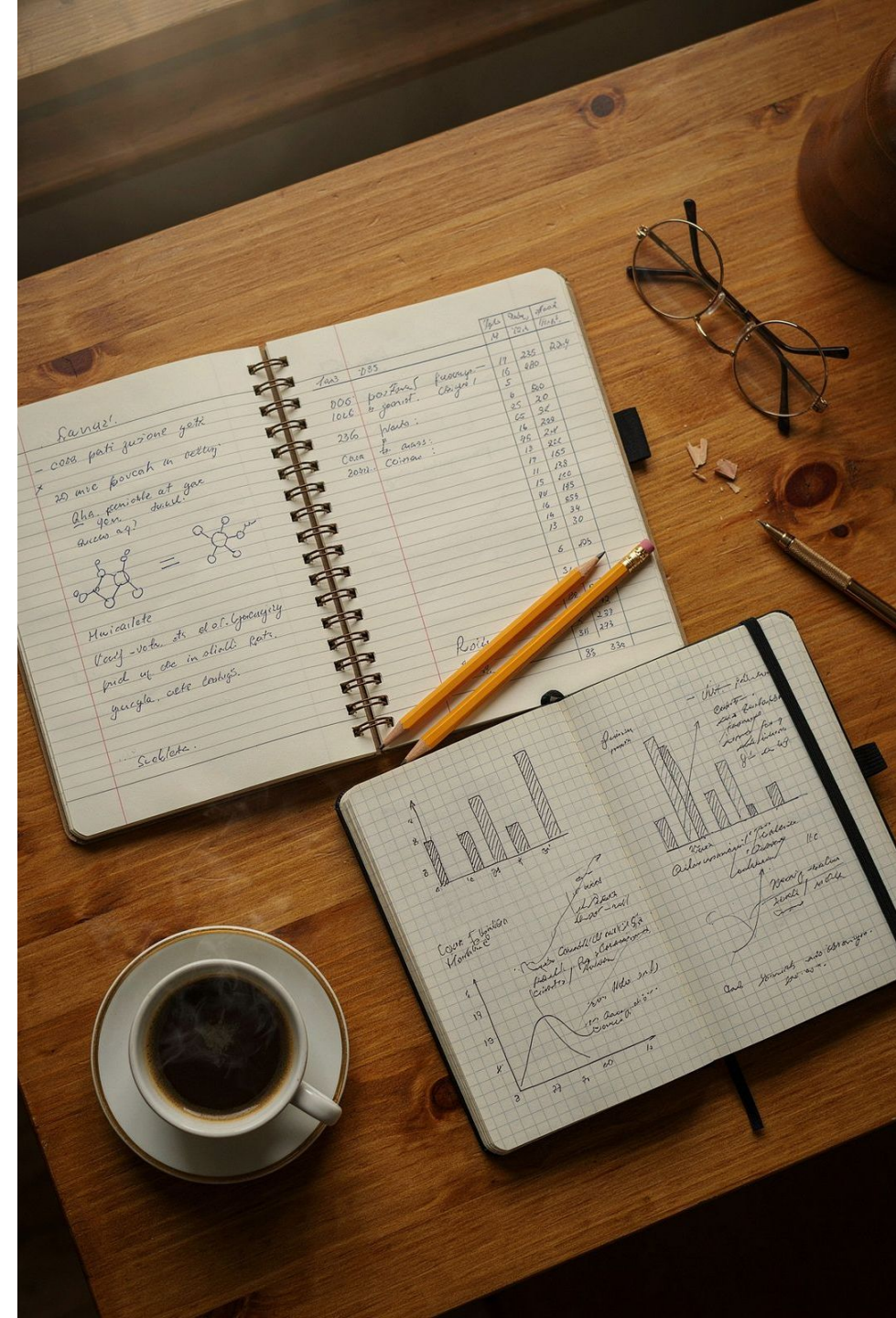


Seguimiento de Experimentos en Machine Learning

Una guía práctica para registrar, versionar y reproducir tus experimentos de ML con rigor científico.

S15.1 · PÍLDORA FORMATIVA



Índice de contenidos

01

Por qué registrar experimentos

La base de la ciencia reproducible

02

Qué registrar

Parámetros, métricas y artefactos

03

Herramientas de seguimiento

MLflow, W&B y soluciones simples

04

Versionado y reproducibilidad

Datos, código, modelos y semillas

05

Comparativa justa

Evitar sesgos y fugas de datos

¿Por qué registrar experimentos?

En ML es habitual lanzar docenas —o cientos— de experimentos antes de encontrar un modelo competitivo. Sin un registro sistemático, es imposible saber **qué configuración produjo qué resultado**, lo que conduce a trabajo duplicado, errores no detectados y resultados irrepetibles.



Reproducibilidad

Repetir exactamente un experimento pasado en cualquier momento



Trazabilidad

Saber qué cambió entre dos versiones de un modelo



Colaboración

Compartir resultados verificables con el equipo



Auditoría

Demostrar el proceso completo ante terceros o reguladores



¿Qué registrar en cada experimento?

Un registro completo cubre cuatro grandes categorías. La regla de oro es: **si cambias algo, regístralo**. Cuanto más metódico sea el registro, más fácil será depurar fallos y replicar éxitos.



Hiperparámetros

Learning rate, batch size, número de capas, épocas, umbral de parada temprana...



Métricas

Accuracy, F1, AUC-ROC, pérdida de entrenamiento y validación por época



Artefactos

Pesos del modelo, gráficas, matrices de confusión, exportaciones ONNX



Metadatos

Fecha, entorno, versión de librerías, hardware (GPU/CPU), autor

MLflow: seguimiento open-source

MLflow es la solución más extendida en entornos corporativos europeos. Ofrece una API Python sencilla para registrar experimentos y una interfaz web para explorarlos. Se puede desplegar localmente o en la nube sin coste de licencia.

`mlflow.log_param()` / `log_metric()`

Registra hiperparámetros y métricas con una sola línea de código

`mlflow.log_artifact()`

Guarda cualquier fichero como artefacto vinculado al experimento

Model Registry

Gestiona el ciclo de vida del modelo: staging, producción, archivado



HERRAMIENTAS

Weights & Biases: colaboración y visualización

Weights & Biases (W&B) está orientado a equipos y proyectos de investigación que necesitan comparar experimentos de forma visual y colaborar en tiempo real. Su interfaz es especialmente potente para el análisis de curvas de aprendizaje y búsquedas de hiperparámetros (Sweeps).

Runs & Projects

Organiza experimentos en proyectos compartidos con el equipo

Sweeps

Búsqueda automática de hiperparámetros (grid, random, Bayesian)

Artifacts

Versionado de datasets y modelos integrado en la plataforma

Reports

Informes interactivos para comunicar hallazgos a stakeholders



La opción mínima viable: un CSV bien estructurado

¿Cuándo usar un CSV?

Cuando el proyecto es pequeño, el equipo trabaja solo o las políticas de la organización impiden usar servicios en la nube. Un CSV es portátil, auditable y no requiere infraestructura.

La clave está en **definir las columnas desde el principio** y comprometerse a rellenarlas tras cada experimento, sin excepciones.

Columnas recomendadas

Campo	Ejemplo
experiment_id	exp_042
fecha	2024-11-15
modelo	RandomForest
hiperparámetros	n_est=200, max_depth=10
val_F1	0.847
git_commit	a3f9c12
notas	Aumenté dropout

Versionado de datos, código y modelos

Los tres pilares de la reproducibilidad completa. No basta con guardar el modelo: si no sabes exactamente con qué datos y con qué código fue entrenado, el experimento no es reproducible.



- Versionado de datos
DVC o snapshots de datasets
- Versionado de código
Git con hash de commit
- Versionado de modelos
MLflow o W&B Artifacts

La práctica recomendada es **registrar el hash del commit de Git junto a cada experimento**, de modo que siempre se pueda recuperar el código exacto que generó un resultado. Para los datos, herramientas como **DVC** permiten versionar conjuntos de datos de gran tamaño sin sobrecargar el repositorio.

Semillas y reproducibilidad estocástica

Los algoritmos de ML dependen de procesos aleatorios: inicialización de pesos, barajado de datos, dropout, splits train/val... Sin fijar la semilla aleatoria, dos ejecuciones del mismo código producirán resultados distintos.

Fijar la semilla global

Establece `random.seed(42)`, `np.random.seed(42)` y `torch.manual_seed(42)` al inicio del script. En TensorFlow usa `tf.random.set_seed(42)`.

Registrar la semilla como parámetro

Trata la semilla como un hiperparámetro más y guárdala en el registro del experimento. Así podrás reproducir exactamente los mismos splits y orden de batches.

Cuidado con el hardware

Las operaciones en GPU pueden ser no deterministas por defecto. En PyTorch activa `torch.use_deterministic_algorithms(True)`, asumiendo un coste moderado de rendimiento.



Evitar la fuga de datos (*data leakage*)

La fuga de datos ocurre cuando información del conjunto de test o del futuro se filtra al entrenamiento. Es uno de los errores más frecuentes y silenciosos: el modelo parece funcionar brillantemente en evaluación... y falla en producción.



Un modelo con métricas demasiado buenas para ser verdad casi siempre indica fuga de datos. Revisa tu pipeline antes de celebrar.



Separar antes de preprocesar

Haz el split train/val/test **antes** de cualquier normalización, imputación o selección de características



Ajustar transformadores solo en train

Aplica `fit` únicamente sobre el conjunto de entrenamiento y luego `transform` en val y test



Usar pipelines de scikit-learn

Los `Pipeline` encapsulan preprocesamiento y modelo, evitando fugas accidentales en cross-validation

Comparativa justa entre modelos

Comparar modelos solo tiene sentido si todos han sido evaluados bajo las **mismas condiciones experimentales**. Pequeñas diferencias en el conjunto de validación, la semilla o el preprocesamiento pueden hacer que un modelo inferior parezca superior.

Mismo conjunto de evaluación Usa exactamente el mismo split de test para todos los modelos comparados, generado con la misma semilla	Mismas métricas Define las métricas de éxito antes de entrenar, no después de ver los resultados
Mismo preprocesamiento Aplica exactamente el mismo pipeline de transformación, ajustado siempre sobre los mismos datos de entrenamiento	Significancia estadística Usa tests como Wilcoxon o bootstrap para determinar si las diferencias son reales o fruto del azar

Comparativa de herramientas de seguimiento

La elección de herramienta depende del tamaño del equipo, los requisitos de privacidad y la infraestructura disponible. Ninguna es universalmente mejor: elige la que reduzca la fricción en tu flujo de trabajo.

Criterio	MLflow	Weights & Biases	CSV
Coste	Gratuito (open-source)	Freemium / Enterprise	Gratuito
Curva de aprendizaje	Media	Baja	Muy baja
Colaboración en equipo	Sí (self-hosted)	Sí (nube)	Limitada
Versionado de modelos	Model Registry	Artifacts	Manual
Privacidad de datos	Alta (local)	Media (nube)	Total
Visualización	Buena	Excelente	Ninguna

Conclusiones clave

- 1** Registra siempre, aunque sea en un CSV
Cualquier registro sistemático es mejor que ninguno. La consistencia importa más que la herramienta.
- 2** Versiona los tres pilares
Datos + código (commit) + modelo. Si falta uno, el experimento no es reproducible.
- 3** Fija y registra la semilla aleatoria
Es el paso más sencillo y más ignorado para garantizar la reproducibilidad.
- 4** Separa datos antes de preprocesar
Es la defensa más efectiva contra la fuga de datos silenciosa.
- 5** Compara modelos bajo las mismas condiciones
Define métricas y splits antes de entrenar. Un experimento justo es la base de una buena decisión.